Algorithm – Part 3: Complexity of Algorithms

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Introduction

- Factors to decide a satisfactory algorithm
  - Correctness: though test cases
  - Efficiency: given a specific input
    - Time used: **time complexity**
    - Amount of memory required: **space complexity**
    - Space complexity will be covered in later courses, so we focus on time complexity.
    - Determine the number of operations, e.g. comparisons, and arithmetic operations.
    - Measurement: big-O and/or big-Theta notations

Complexity Analysis of Algorithms$_1$

- **Example**: Describe the time complexity of the algorithm for finding the maximum element in a finite sequence.

```
procedure max(a_1, a_2, …., a_n: integers)
        max := a_1
        for i := 2 to n
                if max < a_i then max := a_i
        return max
```

- **Solution**: Count the number of basic operations.
  - Each time i is incremented, a test is made to see if $i \leq n$.
  - In each pass of the for loop, there is comparison and an assignment.
  - Exactly $4(n - 1) + 1 = 4n - 3$ operations are made.
- Hence, the time complexity of the algorithm is $\Theta(n)$.

Complexity Analysis of Algorithms$_2$

- We can use this analysis to see whether it is practical to use this algorithm to solve problems with input of a particular size.
- Will the input content influence the execution time?
  - e.g. in linear search, the first element is the one we want: $O(1)$. What if it's the last element? It will be $O(n)$.
- We will focus on the **worst-case time complexity** of an algorithm
  - Average case time complexity is relatively difficult to determine. You will learn in later courses.

Worst-Case Complexity of Linear Search

- **Example**: Determine the time complexity of the linear search algorithm.

	# of operations
procedure *linear search*(x: integer, $a_1, a_2, \ldots, a_n$: distinct integers)	
$i := 1$	1 assignment
while ($i \le n$ and $x \ne a_i$)	2 comparisons/loop
$\quad i := i + 1$	1 addition operation/loop 1 assignment
if $i \le n$ **then** *location* := i	1 comparison and 1 assignment
else *location* := 0	
return *location*	

- For the while loop, if $x = a_i$, $4i$ operations are used. If x is not on the list, $4n$ operations are made. Then an additional if-statement is used to exit the loop.
- For worst case, there are $4n + 3$ comparisons are made.
- Time complexity is $\Theta(n)$.

Average Complexity of Linear Search

- **Example**: Describe the average case performance of the linear search algorithm.
- **Solution**: Assume the element is in the list and that the possible positions are equally likely. By the argument on the previous slide, if $x = ai$, the number of comparisons is $4i + 3$.

$$\frac{3 + 7 + 11 + \cdots + (4n + 3)}{n} = \frac{3n + 4\sum_{i=1}^{n} i}{n}$$

$$= \frac{2n^2 + 5n}{n} = 2n + 5$$

Hence the average-case complexity is $\Theta(n)$.

Worst Case of Binary Search

	# of operations
procedure binary search(x: integer, $a_1, a_2, \ldots, a_n$: increasing integers)	
$i := 1$ {i is the left endpoint of interval}	2 assignments
$j := n$ {j is right endpoint of interval}	
while $i < j$	1 comparison/loop
$m := \lfloor (i + j)/2 \rfloor$	2 arithmetic operations
if $x > a_m$ then $i := m + 1$	1 comparison (and maybe 1 arithmetic operation)
else if $x < a_m$ then $j := m$	
else return *location*	
if $x = a_i$ **then** *location* $:= i$	2 assignments
else *location* $:= 0$	1 comparison
return *location*	1 assignments

- For worst case, there are k iterations, where $2^{k-1} \leq n \leq 2^k$, so $k = \lceil logn \rceil$.
  - In each iteration of the loop, there are at most 7 operations
- Hence, at most $7k + 4 = 7logn + 4$
- Time complexity is $\Theta(logn)$.

Worst Case of Bubble Sort

```
procedure bubblesort(a_1,…,a_n: real numbers
                                with n ≥ 2)
    for i := 1 to n − 1
        for j := 1 to n − i
            if a_j > a_{j+1} then
                interchange a_j and a_{j+1}
```

# of operations

1 arithmetic operation & comparison/1st-loop

1 arithmetic operation & comparison/2nd-loop

1 comparison and 1 interchange operation (3 assignments)

- A sequence of $n-1$ passes is made through the list. On each pass $n - i$ comparisons are made, the interchange operation would be counted in the worst case.

$$\sum_{i=1}^{n-1} 2 * 4(n-i) = 8n(n-1) - 8\frac{n(n-1)}{2} = 4n^2 - 4n$$

- The worst-case complexity of bubble sort is $\Theta(n^2)$.

Worst Case of Insertion Sort

```
procedure insertion sort(a_1,…,a_n: real numbers with n ≥ 2)
        for j := 2 to n
            i := 1
            while a_j > a_i
                i := i + 1
            m := a_j
            for k := 0 to j − i − 1
                a_{j-k} := a_{j-k-1}
            a_i := m
```

# of operations

1 arithmetic operation & 1 comparison/1st-loop

1 assignment

1 comparison/while-loop

1 arithmetic operation & 1 assignment

1 assignment

1 arithmetic operation & 1 comparison /for-loop

1 assignment

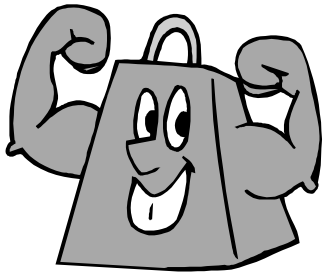
1 assignment

- In worst case, there are $j - 1$ loops in the second for loop.
- The total number of operations in worst case is

$$\sum_{j=2}^{n}(3i + 3(j - i) + 3) = 3\sum_{j=2}^{n}(j + 1) = \frac{3}{2}(n - 1)(n + 4) = \Theta(n^2)$$

- The complexity is $\Theta(n^2)$.

Brute-Force Algorithms



- A *brute-force* algorithm is solved in the most straightforward manner, without taking advantage of any ideas that can make the algorithm more efficient.
- Brute-force algorithms we have previously seen are linear search, bubble sort, and insertion sort.
  - There are better sorting algorithms, e.g. MergeSort, HeapSort, QuickSort, etc.: $\boldsymbol{O(nlogn)}$

Understanding the Complexity of Algorithms 1

- **TABLE 1** Commonly Used Terminology for the Complexity of Algorithms.

Complexity	***Terminology***
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Understanding the Complexity of Algorithms 2

- **TABLE 2** The Computer Time Used by Algorithms.

Problem Size	*Bit Operations Used*					
n	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

- Times of more than 10^{100} years are indicated with an *.

Practice 1

- 1. Each bit operation can be carried out in 10^{-9} second. For an algorithm that requires f(n) bit operations, what is the largest n that the algorithm can solve it within one second?
  - (1) $f(n) = \log(n)$
  - (2) $f(n) = n^2$
  - (3) $f(n) = 2^n$

Practice 2

- 1. Each bit operation can be carried out in 10^{-9} second. For an algorithm that requires f(n) bit operations, what is the largest n that the algorithm can solve it within one second?
  - (1) $f(n) = \log(n)$
    - $\log(n) \cdot 10^{-9} <= 1$ $\log(n) <= 10^9$ $n = 2^{10^9} \approx 10^{3 \cdot 10^8}$
  - (2) $f(n) = n^2$
    - $n^2 \cdot 10^{-9} <= 1$ $n^2 <= 10^9$ $n = 3.16 \cdot 10^4$
  - (3) $f(n) = 2^n$
    - $2^n \cdot 10^{-9} <= 1$ $2^n <= 10^9$ $n = 29$

Practice 3

- 2. What is the time complexity of the below function?

```
void fun (int n) {
  for (int i =0; i < n; i++)
    for (int j = 0; j < i; j++)
      cout << i + j;
}
```

Practice 4

- 2. What is the time complexity of the below function?

```
void fun (int n) {
  for (int i = 0; i < n; i++)
    for (int j = 0; j <= i; j++)
      cout << i + j;
}
```

- $f(n) = 1+2+3+\ldots+n=n(n+1)/2=O(n^2)$

Practice 5

- 3. What is the time complexity of the below function?

```
void fun (int n) {
  for (int i =0; i < n * n; i++)
    for (int j = 0; j < i; j++)
      cout << i + j;
}
```

Practice 6

- 3. What is the time complexity of the below function?

```
void fun (int n) {
  for (int i =0; i < n * n; i++)
    for (int j = 0; j < i; j++)
      cout << i + j;
}
```

- $f(n)=1+2+3+\ldots+n^2= n^2(n^2+1)/2=O(n^4)$

Extension 1

- 1. What is the time complexity of the below function?

```
void fun (int arr[], int n) {
    int i = 0, j = 0;
    for (; i < n; i++)
        while (j < n && arr[i] < arr[j])
            j++;
}
```

O(n)

Extension 2

- 2. What is the time complexity of the below function?

```
void fun (int n) {
  for (int i = 1; i < n; i++)
    for (int j = 1; j < i; j *= 2)
      cout << i + j;
}
```

- $f(n) <= \log(1)+\log(2)+\log(3)+\log(4)+\ldots+\log(n) = \log(n!) \approx n\log(n) - n = O(n\log(n))$